proj3

March 22, 2026

1 Project 3 — Cryptography (CS-GY 6903)

Group members: - Darren Tahe (dt2607@nyu.edu) - Srikanth Akella (ta2728@nyu.edu) - Matthew Bentz (mb9661@nyu.edu) - Matthew Mobijohn (mm7655@nyu.edu)

1.1 Problem 1 - (Mini Block Cipher Implementation)

Explore different modes of operation through manual encryption and decryption.

1.1.1 Task Details

- Use a hypothetical block cipher with a block length of 4, defined as $E_k(b_1b_2b_3b_4) = (b_2b_3b_1b_4)$.
- Convert English plaintext into a bit string using the provided table (A=0000 to P=1111).
- Encrypt the plaintext 'FOO' using the following modes. Convert the final ciphertexts into letters. Show your work.

1.1.2 2a - Encrypt 'FOO' using ECB mode.

Solution First, we must convert 'FOO' into a bit string.

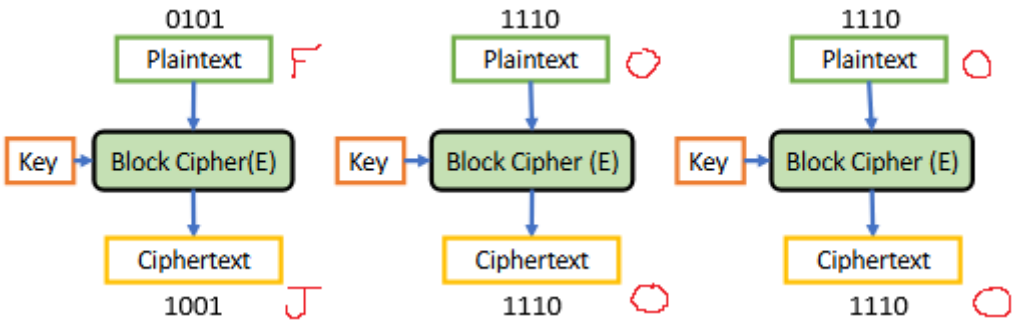
The plaintext mappings for the letters are: - $F = 0101$ - $O = 1110$

Therefore, we have the bit string representation 010111101110.

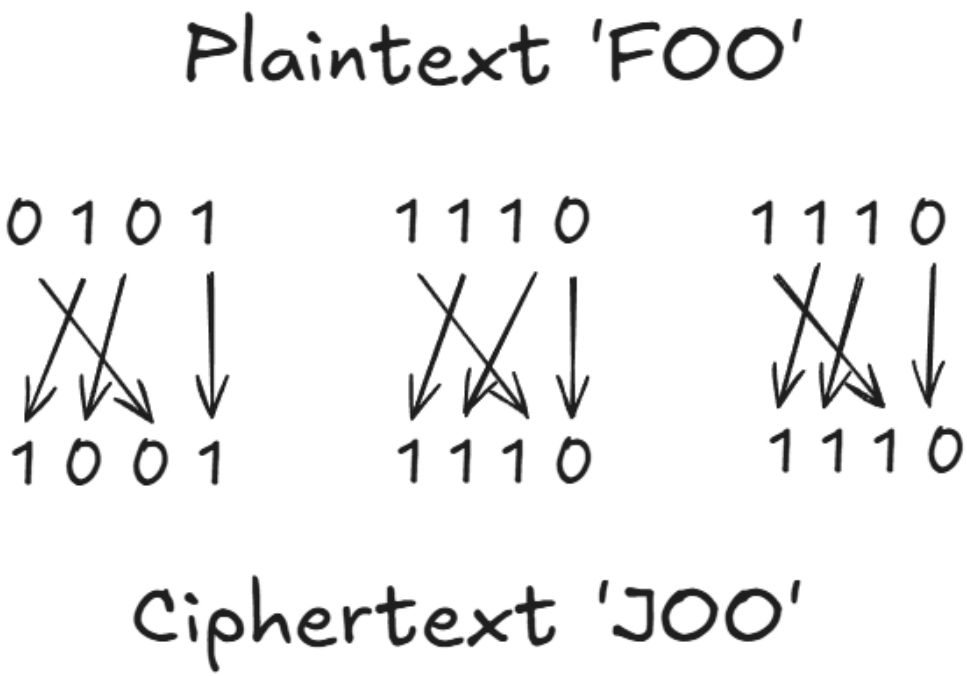
Since our block cipher does not require a key, we can skip the key generation and key expansion functions.

We also do not need to add any padding, since our plaintext has a length of 12, and our blocks are length of 4.

From here, we will begin encryption of the bit string using ECB mode.

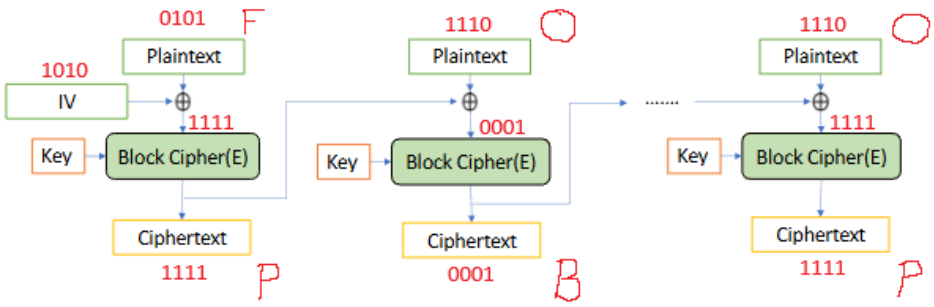


Where our block cipher, defined above, is just a permutation represented as:



1.1.3 2b - Encrypt 'FOO' using CBC mode with IV=1010.

Solution Again, we will use our bit string representation of 'FOO' as 010111101110. And without any key generation, expansion, or padding, we can begin encryption.



Following the steps of the image, we first divide the plaintext into 3 blocks of length 4. With an initialization vector 0101, we take the following actions:

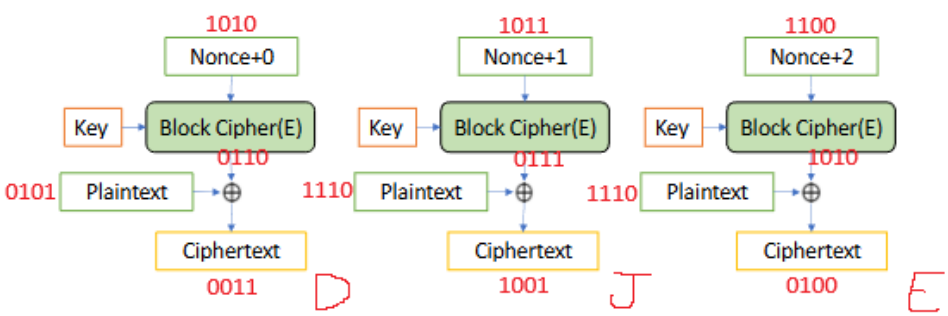
- $F \oplus IV$ or $0101 \oplus 1010 = 1111$
- $E_k(1111) = 1111$ (Ciphertext)
- $O \oplus P$ or $1110 \oplus 1111 = 0001$
- $E_k(0001) = 0001$ (Ciphertext)
- $O \oplus B$ or $1110 \oplus 0001 = 1111$
- $E_k(1111) = 1111$ (Ciphertext)

And the resulting ciphertext produced is PBP

1.1.4 2c - Encrypt 'FOO' using CTR mode with ctr=1010.

Solution Once again, we will use our bit string representation of 'FOO' as 010111101110.

And without any key generation, expansion, or padding, we can begin encryption.



Following the steps of the image, we first divide the plaintext into 3 blocks of length 4. With a nonce value of 1010, we take the following actions:

- Calculate $Nonce + i$
  - 0 - 1010
  - 1 - 1011
  - 2 - 1100
- Encrypt each nonce input
  - 0 - $E_k(1010) = 0110$

  - 1 - $E_k(1011) = 0111$
  - 2 - $E_k(1100) = 1010$
- XOR encrypted nonce with plaintext
  - 0 - $0110 \oplus 0101 = 0011$
  - 1 - $0111 \oplus 1110 = 1001$
  - 2 - $1010 \oplus 1110 = 0100$

And the resulting ciphertext produced is DJE

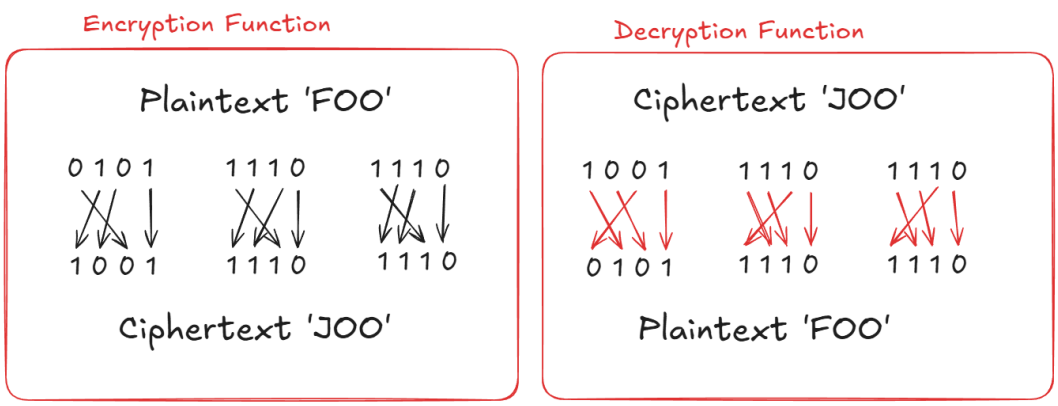
1.1.5 3a - Manually decrypt each ciphertext from 2a/2b/2c.

Solution We have the following ciphertexts to decrypt: - JOO w/ ECB mode - PBP w/ CBC mode - DJE w/ CTR mode

We also need to define our decryption function, $(E_k(c))^{-1}$, as the inverse of the encryption function, $E_k(m)$.

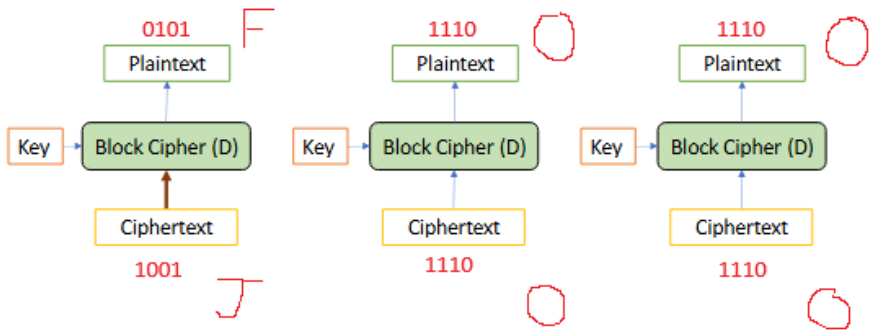
Since $E_k(b_1b_2b_3b_4) = (b_2b_3b_1b_4)$

Our decryption function is defined as $D_k(b_1b_2b_3b_4) = (b_3b_1b_2b_4)$. A visual representation of the block cipher's inverse is shown below.



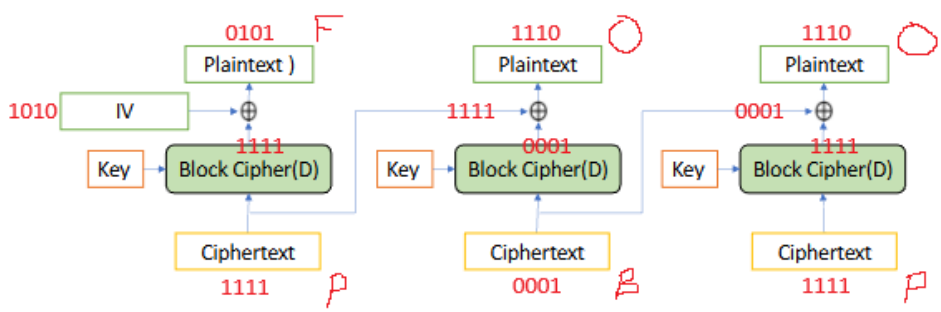
Using this decryption function, we can begin our decryptions.

1.2 'JOO' w/ ECB mode



Where: - $D(J) = F$ - $D(O) = O$ - $D(O) = O$

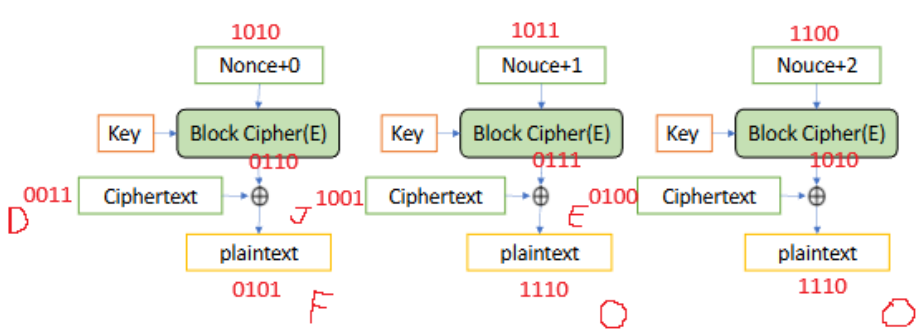
1.3 'PBP' w/ CBC mode



Where: - $D(P) = 1111$ - $D(B) = 0001$ - $D(P) = 1111$

Then: - $1111 \oplus 1010(IV) = 0101(F)$ - $0001 \oplus 1111(P) = 1110(O)$ - $1111 \oplus 0001(B) = 1110(O)$

1.4 'DJE' w/ CTR mode



Where: $E(IV_i)$ - $E(1010) = 0110$ - $E(1011) = 0111$ - $E(1100) = 1010$

Then: $c_i \oplus E(IV_i)$ - $0011 \oplus 0110 = 0101(F)$ - $1001 \oplus 0111 = 1110(O)$ - $0100 \oplus 1010 = 1110(O)$

1.5 Problem 2 - (Implementing MITM Attack on a Block Cipher)

1.5.1 Task 1 Overview:

Implement a block cipher based on Simplified AES.

1.5.2 Pseudo-code:

```
1. Get (Key1, Key2) from Key K using the key expansion.
2. Two rounds of Encrypt to get the ciphertext.
    a. encrypt_round1() using K1
        i.   Subsititute()
        ii.  Shift()
        iii. Mix()
        iv.  AddRoundKey()
    b. encrypt_round2() using K2
        i.   Subsititute()
        ii.  Shift()
        iii. AddRoundKey()
3. Two rounds of decryption to recover the plaintext
    a. decrypt_round2() using K2
        i.   AddRoundKey()
        ii.  Shift()
        iii. Subsititute()
    b. decrypt_round1() using K1
        i.   AddRoundKey()
        ii.  Mix()
        iii. Shift()
        iv.  Subsititute()
```

1.5.3 Task 1a - Mini Block Cipher encryption and decryption functions.

Implement the Mini Block Cipher functions with 16-bit key and block size.

Solution

[284]:
```
# ----------
# S-box(nib)

sbox = {
    0x0:0x9, 0x8:0x6,
```

```python
    0x1:0x4, 0x9:0x2,
    0x2:0xA, 0xA:0x0,
    0x3:0xB, 0xB:0x3,
    0x4:0xD, 0xC:0xC,
    0x5:0x1, 0xD:0xE,
    0x6:0x8, 0xE:0xF,
    0x7:0x5, 0xF:0x7
}

inv_sbox = {v:k for k,v in sbox.items()}

# -------------

def key_expansion(key: int) -> tuple[int, int, int]:
    """
    SAES Key Expansion for a 16-bit key.

    Args:
        key (int): A 16-bit integer representing the original key.

    Returns:
        tuple: A tuple containing the two round keys (Key0, Key1, Key2).
    """
    k0 = key >> 12 & 0xF
    k1 = key >> 8 & 0xF
    k2 = key >> 4 & 0xF
    k3 = key & 0xF

    # Extract w0 = K0||K1 and w1 = K2||K3 from the 16-bit key
    w0 = k0 << 4 | k1
    w1 = k2 << 4 | k3

    # First round constant is essentially not used; defined for simplicity
    RCON = [0x00, 0x80, 0x30]

    # Substitution using the S table
    def SubNib(nib):
        return sbox[nib >> 4 & 0xF] << 4 | sbox[nib & 0xF]

    # One byte left circular left shift
    def RotNib(nib):
        return ((nib << 4) | (nib >> 4)) & 0xFF

    w2 = w0 ^ RCON[1] ^ SubNib(RotNib(w1))
    w3 = w1 ^ w2

    w4 = w2 ^ RCON[2] ^ SubNib(RotNib(w3))
```

```
    w5 = w3 ^ w4

    return w0 << 8 | w1, w2 << 8 | w3, w4 << 8 | w5
```

[285]:
```
def substitute(state: int, sbox: dict[hex, hex]) -> int:
    """
    Substitute 4-bit nibbles from the defined S-box table.
    """
    result = sbox[state >> 12 & 0xF] << 12 | \
             sbox[state >> 8 & 0xF] << 8 | \
             sbox[state >> 4 & 0xF] << 4 | \
             sbox[state & 0xF]
    return result
```

[286]:
```
def shift(state: int) -> int:
    """
    Shift rows for a 2x2 state representation

    Input:
    [ 0  2 ]
    [ 1  3 ]

    Output:
    [ 0  2 ]
    [ 3  1 ]
    """
    row0 = state & 0xF0F0
    row1 = state & 0x0F0F
    row1_shifted = ((row1 << 8) | (row1 >> 8)) & 0x0F0F
    return row0 | row1_shifted
```

[287]:
```
def mix(state: int) -> int:
    """
    Mix columns for a 2x2 state representation
    """
    # column(1,2), row1 = (b0 ^ b6) (b1 ^ b4 ^ b7) (b2 ^ b4 ^ b5) (b3 ^ b5)
    # column(1,2), row2 = (b2 ^ b4) (b0 ^ b3 ^ b5) (b0 ^ b1 ^ b6) (b1 ^ b7)
    def mix_column(col: int) -> int:
        b = [(col >> (7 - i)) & 1 for i in range(8)]

        out = [
            b[0] ^ b[6],
            b[1] ^ b[4] ^ b[7],
            b[2] ^ b[4] ^ b[5],
            b[3] ^ b[5],
            b[2] ^ b[4],
            b[0] ^ b[3] ^ b[5],
```

```
            b[0] ^ b[1] ^ b[6],
            b[1] ^ b[7],
        ]

        result = 0
        for bit in out:
            result = (result << 1) | bit
        return result

    col1 = (state >> 8) & 0xFF   # high byte
    col2 = state & 0xFF          # low byte

    return (mix_column(col1) << 8) | mix_column(col2)

def inv_mix(state: int) -> int:
    """
    Inverse mix columns for a 2x2 state representation
    """
    # column(1,2), row1 = (b3 ^ b5) (b0 ^ b6) (b1 ^ b4 ^ b7) (b2 ^ b3 ^ b4)
    # column(1,2), row2 = (b1 ^ b7) (b2 ^ b4) (b0 ^ b3 ^ b5) (b0 ^ b6 ^ b7)
    def mix_column(col: int) -> int:
        b = [(col >> (7 - i)) & 1 for i in range(8)]

        out = [
            b[3] ^ b[5],
            b[0] ^ b[6],
            b[1] ^ b[4] ^ b[7],
            b[2] ^ b[3] ^ b[4],
            b[1] ^ b[7],
            b[2] ^ b[4],
            b[0] ^ b[3] ^ b[5],
            b[0] ^ b[6] ^ b[7],
        ]

        result = 0
        for bit in out:
            result = (result << 1) | bit
        return result

    col1 = (state >> 8) & 0xFF   # high byte
    col2 = state & 0xFF          # low byte

    return (mix_column(col1) << 8) | mix_column(col2)
```

[288]:
```
def add_round_key(state: int, key: int) -> int:
    return state ^ key
```

[289]:
```python
# ----------
# Encryption

def encrypt_round1(state: int, K1: int) -> int:
    state = substitute(state, sbox)
    state = shift(state)
    state = mix(state)
    state = add_round_key(state, K1)

    return state

def encrypt_round2(state: int, K2: int) -> int:
    state = substitute(state, sbox)
    state = shift(state)
    state = add_round_key(state, K2)

    return state

def encrypt(plaintext: int, key: int) -> int:
    _, K1, K2 = key_expansion(key)
    state = encrypt_round1(plaintext, K1)
    ciphertext = encrypt_round2(state, K2)

    return ciphertext

# ----------
# Decryption

def decrypt_round2(state: int, K2: int) -> int:
    state = add_round_key(state, K2)
    state = shift(state)
    state = substitute(state, inv_sbox)

    return state

def decrypt_round1(state: int, K1: int) -> int:
    state = add_round_key(state, K1)
    state = inv_mix(state)
    state = shift(state)
    state = substitute(state, inv_sbox)

    return state

def decrypt(ciphertext: int, key: int) -> int:
    _, K1, K2 = key_expansion(key)
    state = decrypt_round2(ciphertext, K2)
    plaintext = decrypt_round1(state, K1)
```

```
    return plaintext
```

[290]:
```
# Example encrypting a block. (ASCII ZC)
plaintext = 0b0101101001000011
key = 0b0101100101111010
print(f"Plaintext: \t\t{plaintext:016b}")
ciphertext = encrypt(plaintext, key)
print(f"Ciphertext: \t\t{ciphertext:016b}")
decrypted_ciphertext = decrypt(ciphertext, key)
print(f"Decrypted Ciphertext: \t{decrypted_ciphertext:016b}")

print()

# Sanity check from slides example, with add round key before first round.
K0, K1, K2 = key_expansion(0b0101100101111010)

plaintext = add_round_key(plaintext, K0)
ciphertext = encrypt(plaintext, key)
print(f"Ciphertext (slides): \t{ciphertext:016b}")

decrypted_ciphertext = decrypt(ciphertext, key)
decrypted_ciphertext = add_round_key(decrypted_ciphertext, K0)
print(f"Decrypted Ciphertext: \t{decrypted_ciphertext:016b}")
```

```
Plaintext:              0101101001000011
Ciphertext:             1110100110010001
Decrypted Ciphertext:   0101101001000011

Ciphertext (slides):    1010100111110110
Decrypted Ciphertext:   0101101001000011
```

1.5.4 Task 1b - Create 10 distinct plaintext-ciphertext pairs using the functions.

Make at least ten pairs of plaintexts and ciphertexts using the above implementation.

Solution

[291]:
```
import random

key = random.getrandbits(16)
print("Key: \t\t", f"{key:016b}")
print()

pairs = []
for i in range(10):
```

```python
    plaintext = random.getrandbits(16)
    ciphertext = encrypt(plaintext, key)
    decrypted = decrypt(ciphertext, key)
    pairs.append((plaintext, ciphertext))

    print("Plaintext:  \t", f"{plaintext:016b}")
    print("Ciphertext: \t", f"{ciphertext:016b}")
    print("Decrypted: \t", f"{decrypted:016b}")
    print()
```

```
Key:            1110110110011100

Plaintext:      0100101001011110
Ciphertext:     1101000110110011
Decrypted:      0100101001011110

Plaintext:      0100001101000110
Ciphertext:     0110011011001111
Decrypted:      0100001101000110

Plaintext:      0110000010111101
Ciphertext:     0101110010110010
Decrypted:      0110000010111101

Plaintext:      1100110101000011
Ciphertext:     1110100010100011
Decrypted:      1100110101000011

Plaintext:      1010000001001001
Ciphertext:     1000011111011001
Decrypted:      1010000001001001

Plaintext:      0100110111101000
Ciphertext:     1110100101010001
Decrypted:      0100110111101000

Plaintext:      0011001111111111
Ciphertext:     1101000111101101
Decrypted:      0011001111111111

Plaintext:      0100001111000101
Ciphertext:     1001101000000000
Decrypted:      0100001111000101

Plaintext:      0011101100111000
Ciphertext:     0010010011001100
Decrypted:      0011101100111000
```

```
Plaintext:       0011100100110010
Ciphertext:      0101011110000000
Decrypted:       0011100100110010
```

1.5.5 Task 2 Overview:

Implement a MITM Attack.

1.5.6 Pseudo-code:

```
a. Calculate X = encrypt_round1(K1, P)
b. Calculate X' = decrypt_round2(K2, C).
c. Find out a pair (K1, K2) such at X = X'
d. For one specific (P, C), there might be many matched pairs, we need to use more plaintext a
e. We can recover the plaintext from any ciphertext by using the key pair. In other words, we
```

1.5.7 Task 2a - Attack strategy implementation.

Implementation of meet-in-the-middle attack strategy on the mini block cipher based on the above strategy a-d.

Solution

[292]:
```python
def mitm_attack(pairs: list[tuple[int, int]]) -> list[tuple[int, int]]:
    """
    Meet-in-the-middle attack on the mini block cipher.

    Steps:
        a. For all 2^16 possible K1, compute X = encrypt_round1(P, K1)
           and store in a lookup table: X -> [K1, ...]
        b. For all 2^16 possible K2, compute X' = decrypt_round2(C, K2)
           and check if X' is in the table (i.e. X == X').
        c. Any (K1, K2) match is a candidate key pair.
        d. Validate candidates against all remaining (P, C) pairs
           to eliminate false positives.

    Args:
        pairs: list of (plaintext, ciphertext) pairs

    Returns:
        list[tuple]: List of (K1, K2) candidate pairs.
    """

    P0, C0 = pairs[0]
```

```python
    print("Using the following plaintext/ciphertext pair:")
    print(f"\tPlaintext:  {P0:016b}")
    print(f"\tCiphertext: {C0:016b}")
    print()

    print("a. Calculating X = encrypt_round1(K1, P)...")
    forward_table: dict[int, list[int]] = {}
    for K1 in range(2**16):
        X = encrypt_round1(P0, K1)
        if X not in forward_table:
            forward_table[X] = []
        forward_table[X].append(K1)
    print(f"\tCreated lookup table with {len(forward_table)} encrypted values.")

    print("b. Calculating X' = decrypt_round2(K2, C)...")
    candidates: list[tuple[int, int]] = []
    for K2 in range(2**16):
        X_prime = decrypt_round2(C0, K2)

        # c. Find out a pair (K1, K2) such that X = X'
        if X_prime in forward_table:
            for K1 in forward_table[X_prime]:
                candidates.append((K1, K2))

    print(f"\tCandidates after first pair: {len(candidates)}")

    print("d. Eliminating false positives with additional pairs...")
    for P, C in pairs[1:]:
        survivors = []
        for K1, K2 in candidates:
            mid = encrypt_round1(P, K1)
            if encrypt_round2(mid, K2) == C:
                survivors.append((K1, K2))

        candidates = survivors
        print(f"\tCandidates remaining after filtering: {len(candidates)}")
        if len(candidates) == 1:
            break

    return candidates
```

1.5.8 Task 2b - Attack strategy execution and presentation.

Rubric Clear presentation of meet-in-the-middle attack results using plaintext-ciphertext pairs from Task 1b.

---

[294]:
```python
candidate_key_pairs = mitm_attack(pairs)
if (len(candidate_key_pairs) == 1):
    key_pair = candidate_key_pairs[0]
    print("Found key pair: ", key_pair)
    print()

    print("Using found key pair to decrypt all ciphertexts from task 1b...")
    for i, (P, C) in enumerate(pairs):
        decrypted = decrypt_round1(decrypt_round2(C, key_pair[1]), key_pair[0])
        ok = (decrypted == P)
        print(f"\tPair {i+1:02d}: P={P:016b}  C={C:016b}  decrypted={decrypted:
↪016b}  {' ' if ok else ' '}")
```

```
Using the following plaintext/ciphertext pair:
        Plaintext:  0100101001011110
        Ciphertext: 1101000110110011

a. Calculating X = encrypt_round1(K1, P)…
        Created lookup table with 65536 encrypted values.
b. Calculating X' = decrypt_round2(K2, C)…
        Candidates after first pair: 65536
d. Eliminating false positives with additional pairs…
        Candidates remaining after filtering: 32
        Candidates remaining after filtering: 8
        Candidates remaining after filtering: 4
        Candidates remaining after filtering: 1
Found key pair:  (44851, 9239)

Using found key pair to decrypt all ciphertexts from task 1b…
        Pair 01: P=0100101001011110  C=1101000110110011
decrypted=0100101001011110
        Pair 02: P=0100001101000110  C=0110011011001111
decrypted=0100001101000110
        Pair 03: P=0110000010111101  C=0101110010110010
decrypted=0110000010111101
        Pair 04: P=1100110101000011  C=1110100010100011
decrypted=1100110101000011
        Pair 05: P=1010000001001001  C=1000011111011001
decrypted=1010000001001001
        Pair 06: P=0100110111101000  C=1110100101010001
decrypted=0100110111101000
        Pair 07: P=0011001111111111  C=1101000111101101
decrypted=0011001111111111
        Pair 08: P=0100001111000101  C=1001101000000000
decrypted=0100001111000101
        Pair 09: P=0011101100111000  C=0010010011001100
decrypted=0011101100111000
        Pair 10: P=0011100100110010  C=0101011110000000
```

```
decrypted=0011100100110010
```

1.5.9 Task 3 Overview:

Analyze the time and memory complexity of the attack compared with the naive exhaustive key search.

1.5.10 Task 3a

What is the key space for the mini block cipher?

Solution The mini block cipher has a key size of 16 bits, therefore, the key space is $2^{16} = 65536$ possible keys.

An attacker performing a brute-force attack would need to try up to 65,536 keys to find the correct one.

1.5.11 Task 3b

Imagine the mini block cipher is executed twice to generate a cipher text. It is called double mini cipher block. We need a key in 32 bits. The first 16 to the first mini block cipher, the remaining 16 to the second mini block cipher. The meet in the middle attack is to match the state for the first encryption of mini block cipher and the second decryption mini block. How many operations are needed to such attack?

Solution For MITM attack, instead of testing all 2^{32} keys, we split it into 2 parts.

2^{16} for encryptions and 2^{16} for decryptions which is $2 * 2^{16} \approx 2^{17} \approx 131,072$ operations.

1.5.12 Task 3c

If we do exhaustive key search for the double mini block cipher, how many operations are needed?

Solution For double mini cipher block, the total key space becomes 2^{32}.

A naive brute force (exhaustive key search) attack would require $2^{32} = 4,294,967,296$ operations, the equivalent to the possible number of keys.

1.5.13 Task 3d

What is the trade off in this attack?

Solution The Meet-in-the-Middle attack significantly reduces the computational complexity of breaking a double encryption scheme from 2^{32} operations to approximately 2^{17} operations by storing intermediate encryption results, at the cost of increased memory usage.

The increased memory cost comes from having to store the lookup table for X = encrypt_round1(), as seen in the implementation. Typically, the memory cost of the lookup table for finding intermediary values is much more viable than the computational cost required to brute force an exponential amount of possible keys, and is therefore more effective.

1.5.14 Task 4 Overview:

Discuss how we can improve the mini block cipher.

Students may start by analyzing whether the SAES is vulnerable to the MITM attack or not. Remember SAES is the mini block cipher except that it has the initial addoundkey() using the original key K.

Solution The mini block cipher is technically exploitable via Meet-in-the-Middle attacks because it has two independent rounds of encryption, which allows an attacker to bypass the full encryption scheme and only utilize the middle state. Regardless, the key expansion does not produce two 8-bit keys, and therefore it's computationally useless to perform. An exhaustive key search for both S-AES and the mini block cipher is equivalent to 2^{16} operations which is trivial.

If we were to attack the mini block cipher with a MITM attack, however, it would be beneficial to include the AddRoundKey operation that S-AES implements. This is because when calculating the lookup table for encrypted values, we now must have the original key value to modify the plaintext before computating the round 1 encyryption with K1. This effectively mitigates the MITM attack, due to the utilization of an additional key.

```
# Encryption with mini-block cipher
# middle state defined as encrypt_round1(K1, P), only requiring K1 to generate.
C = encrypt_round2(K2, encrypt_round1(K1, P))

# Encryption with S-AES
# middle state defined as encrypt_round1(K1, P ^ K0), requiring both K0/K1 to generate, or K0*
C = encrypt_round2(K2, encrypt_round1(K1, P ^ K0))
```

In order to improve the current implementation of the mini block cipher, the best thing to do would be to increase the key size from 16 bits. This would not technically affect the feasability of a MITM attack, though.

To improve the mini block cipher, in relation to MITM attacks: 1. Add an initial AddRoundKey operation 2. Increase the number of rounds 3. Use a stronger key expansion algorithm 4. Improve the mixing operation

These changes would significantly increase the cipher's resistance to cryptographic attacks.